

Cognizant Digital Nurture 5.0

ReactJS Hands-On Exercise 16

React Forms Validation

Objective

The objective of this exercise is to understand and implement **React Forms Validation** by developing a user registration application. The application validates user inputs such as **Name**, **Email**, and **Password** before allowing form submission. This exercise demonstrates the use of controlled components, event handling, form validation, and error handling in React. The validation rules are:

- Name should contain at least **5 characters**.
- Email should contain **@** and **.**
- Password should contain at least **8 characters**.

The application validates the entered data and displays appropriate messages for invalid input or confirms successful submission when all fields satisfy the validation criteria.

Theory

What is React Form Validation?

React Form Validation is the process of checking whether user input satisfies predefined rules before submitting the form. Validation helps ensure that only correct and meaningful information is accepted.

React performs validation using **controlled components**, where the values entered by the user are stored in the component's state and validated through event handlers.

Controlled Components

A controlled component is a form element whose value is controlled by React State.

Whenever the user enters data into a textbox, textarea, or password field, the **onChange()** event updates the component state.

Advantages

- Easy validation of user input.

- Better control over form fields.
- Improved user experience.
- Simplifies error handling.
- Prevents invalid data submission.

Form Validation Rules

The registration form validates the following fields:

Name Validation

- Name should contain at least **5 characters**.

Example

```
if (value.length < 5) {  
  errors.fullName =  
    "Full Name must be at least 5 characters long!";  
}
```

Email Validation

The email should contain a valid email format.

Example

```
const validEmailRegex =  
/^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

Password Validation

Password should contain at least **8 characters**.

Example

```
if (value.length < 8) {  
  errors.password =  
    "Password must be at least 8 characters long!";  
}
```

Event Handling

React provides several event handlers for processing user interactions.

The registration form uses:

- **onChange()** – Updates the state whenever the user types.
- **onSubmit()** – Validates the form before submission.

Validation Process

1. User enters the Name.
2. User enters the Email.
3. User enters the Password.
4. The **handleChange()** method validates each field.
5. The **handleSubmit()** method checks whether all fields are valid.
6. If validation is successful, a success message is displayed.
7. Otherwise, an appropriate error message is shown.

Prerequisites

- Node.js
- npm
- Visual Studio Code
- ReactJS

Software Requirements

- React
 - JavaScript (ES6)
 - HTML
 - CSS
-

Implementation Steps

Step 1

Create a React application.

```
npx create-react-app mailregisterapp
```

Step 2

Navigate to the project folder.

```
cd mailregisterapp
```

Step 3

Create the following files inside the **src** folder.

- Register.js
 - App.js
 - index.js
-

Step 4

Initialize the state variables.

Create state variables for:

- Full Name
 - Email
 - Password
 - Error Messages
-

Step 5

Implement the **handleChange()** method.

The method validates each field whenever the user enters data.

Step 6

Implement the **validateForm()** method.

The method checks whether all validation errors are empty.

Step 7

Implement the **handleSubmit()** method.

The method validates the form and displays:

- **"Valid Form"** if all validations are successful.
 - Appropriate error messages if validation fails.
-

Step 8

Display the registration form.

The form contains:

- Name Textbox
 - Email Textbox
 - Password Textbox
 - Submit Button
-

Step 9

Run the application.

npm start

Project Structure

mailregisterapp

```
|— src
|   |— Register.js
|   |— App.js
|   |— index.js
|   |— index.css
```

|
├─ public
├─ package.json
└─ node_modules

Source Code

Register.js

```
import React, { Component } from "react";  
  
class Register extends Component {  
  constructor() {  
    super();  
    this.state = {  
      fullName: "Rekha Sakipalli",  
      email: "rekhasakipalli@gmail.com",  
      password: "Rekha@123",  
      errors: {  
        fullName: "",  
        email: "",  
        password: ""  
      }  
    };  
  }  
  
  handleChange = (event) => {  
    const { name, value } = event.target;  
    let errors = this.state.errors;  
    switch (name) {  
      case "fullName":  
        errors.fullName =
```

```

        value.length < 5

        ? "Full Name must be at least 5 characters long."

        : "";

    break;

case "email":

    const validEmailRegex =

        /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    errors.email =

        validEmailRegex.test(value)

        ? ""

        : "Email is not valid.";

    break;

case "password":

    errors.password =

        value.length < 8

        ? "Password must be at least 8 characters long."

        : "";

    break;

default:

    break;

}

this.setState({

    errors,

    [name]: value

});

};

validateForm = () => {

    const { errors } = this.state;

```

```
let valid = true;

Object.values(errors).forEach(error => {

  if (error.length > 0) {

    valid = false;

  }

});

return valid;

};

handleSubmit = (event) => {

  event.preventDefault();

  if (this.validateForm()) {

    alert("Registration Successful");

  }

  else {

    alert("Please enter valid details.");

  }

};

render() {

  return (

    <div style={{ textAlign: "center", marginTop: "40px" }}>

      <h1 style={{ color: "red" }}>

        Register Here!!!

      </h1>

      <form onSubmit={this.handleSubmit}>

        <table align="center">

          <tbody>

            <tr>

              <td>Name</td>
```



```
<td>

  <input

    type="text"

    name="fullName"

    value={this.state.fullName}

    onChange={this.handleChange}

  />

</td>

</tr>

<tr>

  <td>Email</td>

  <td>

    <input

      type="email"

      name="email"

      value={this.state.email}

      onChange={this.handleChange}

    />

  </td>

</tr>

<tr>

  <td>Password</td>

  <td>

    <input

      type="password"

      name="password"

      value={this.state.password}

      onChange={this.handleChange}

    />

  </td>

</tr>

</tbody>

</table>
```

```

        />
      </td>
    </tr>
    <tr>
      <td></td>
      <td>
        <button type="submit">
          Submit
        </button>
      </td>
    </tr>
  </tbody>
</table>
</form>
</div>
);
}
}
export default Register;

```

App.js

```

import React from "react";
import Register from "../Register";
function App() {
  return (
    <div>
      <Register />
    </div>
  );
}

```

```
    </div>

  );
}

export default App;
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <React.StrictMode>

    <App />

  </React.StrictMode>
);
```

index.css (Optional)

```
body {
  font-family: Arial, Helvetica, sans-serif;
  background-color: #f5f5f5;
}

table {
  border-spacing: 10px;
}

input {
```

```
padding: 6px;
width: 250px;
}
button {
padding: 8px 18px;
background-color: #007bff;
color: white;
border: none;
cursor: pointer;
}
button:hover {
background-color: #0056b3;
}
```

Output Screenshot:

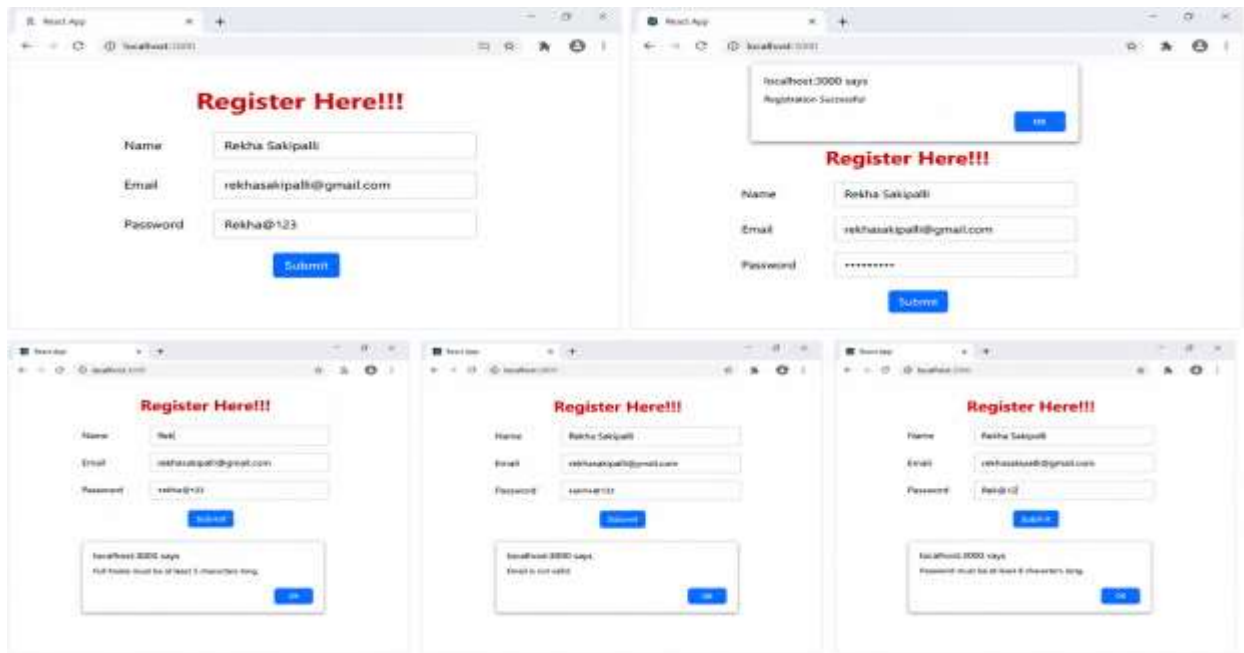
When the application starts, the browser displays a registration form.

The form contains:

- Name Textbox
- Email Textbox
- Password Textbox
- Submit Button

The fields are pre-filled with:

- **Name:** Rekha Sakipalli
- **Email:** rekhasakipalli@gmail.com
- **Password:** Rekha@123



If any field contains invalid input, the application displays an appropriate validation message, such as:

- Full Name must be at least 5 characters long.
- Email is not valid.
- Password must be at least 8 characters long.

These validations match the requirements specified in the exercise.

Learning Outcome

After completing this exercise, I learned how to:

- Create forms using React.
- Implement controlled components.
- Validate user input using event handlers.
- Handle form submission using `onSubmit()`.
- Display validation messages.
- Manage component state effectively.
- Improve the reliability of user input through client-side validation.

Conclusion

This exercise provided practical experience in implementing **React Form Validation** using controlled components and event handling. The registration form demonstrated how to validate user inputs such as name, email, and password before processing the form submission. By using React state, onChange() events, and validation logic, the application ensures that only valid information is accepted. This exercise strengthened the understanding of form validation techniques, state management, and user interaction handling, which are essential skills for developing reliable and user-friendly React applications.